

Explorando Estruturas de Dados Estáticas

Autor: Lucas Matheus dos Santos Marques

Matheus Barbosa dos Santos Carvalho

Victor Carvalho Cedraz

Vitor Hugo da Silva Oliveira

Agenda

O objetivo dessa apresentação é detalhar a modelagem e a implementação do motor de dados estáticos do MVP da MecânicaQA, utilizando arrays unidimensionais para as entidades Peça e Serviço.

1. Contexto do Problema:

Resumo: A MecânicaQA, a base de clientes crescendo e o objetivo do MVP.

2. Modelagem dos TADs:

Resumo: As entidades Peça e Serviço, tratadas como "structs" em Java.

3. O Motor de Armazenamento:

Resumo: Arrays unidimensionais de tamanho fixo no Gerenciador.java.

4. CRUD Estático:

Resumo: Inserção, Busca Linear, Atualização e Remoção sem "buracos".

5. Limitações da Estrutura:

Resumo: O custo $O(n)$ da busca e o limite físico dos arrays.

Contexto do Problema

- A MecâniQA - Automotive Tech é uma startup de Feira de Santana (BA) que oferece uma solução de gestão para lojas de autopeças, auto centers e oficinas mecânicas.
- Com a base de clientes crescendo, a empresa precisa construir a primeira versão (MVP) do sistema interno de gerenciamento de estoque e serviços.
- O Diretor de Engenharia determinou que o núcleo do sistema gerencie a memória de forma manual e contígua — ou seja, sem coleções dinâmicas.
- Objetivos da sprint:
 - Compreender e implementar Tipos Abstratos de Dados (TAD)
 - Dominar a manipulação de listas estáticas (arrays)
 - Implementar algoritmos de busca linear
 - Vivenciar as limitações das estruturas estáticas

Modelagem dos TADs: Peça e Serviço

- As entidades são tratadas como "structs" do C: apenas atributos públicos, sem getters, setters ou métodos internos.
 - Peça: codigo (int), nomePeca (String), nomeFabricante (String), precoCusto (double), precoVenda (double), quantidade (int).
 - Servico: codigo (int), descricao (String), tempoMinutos (int), valor (double).
- O encapsulamento acontece pela criação de um novo objeto (ex.: `Peca peca1 = new Peca();`); os atributos são públicos para permitir a criação de instâncias em outras classes, como a Main.

```
public class Peca { 19 usages  @ Matheus Carvalho +1
    public int codigo; 3 usages
    public String nomePeca; 3 usages
    public String nomeFabricante; 1 usage
    public double precoCusto; 1 usage
    public double precoVenda; 3 usages
    public int quantidade; 3 usages
```

O Motor de Armazenamento

- Arrays unidimensionais de tamanho fixo residem em Gerenciador.java como campos public static.
 - MAX_PECAS = 100 e MAX_SERVICOS = 50 definem a capacidade máxima de cada array.
 - Peca[] pecas = new Peca[MAX_PECAS]; Servico[] servicos = new Servico[MAX_SERVICOS];
- Uma variável auxiliar de controle guarda a próxima posição livre de cada array, evitando percorrer a lista inteira a cada inserção.

```
private static final int MAX_PECAS = 100; 3 usages
public static Peca[] pecas = new Peca[MAX_PECAS]; 15 usages
public static int proximoIndex = 0; 13 usages
```

A variável proximoIndex guarda a informação da primeira posição vazia.

CRUD Estático no Gerenciador

- Toda a lógica de negócio vive em métodos public static, manipulando diretamente os arrays.
 - **Create** — `inserirPeca(Peca nome)`: adiciona o registro no primeiro espaço vazio e incrementa o controle de quantidade.
 - **Read** — `buscarPeca( int codigo)`: busca linear que retorna o índice do elemento ou -1 se não encontrado.
 - **Update** — `mudarPeca(int codigoBusca, String novoNome, double novoValorVenda, int novaQuantidade)`
: chama a busca linear para achar o índice antes de sobrescrever os atributos.
 - **Delete** — `deletarPeca(int codigo)`: reorganiza os elementos subsequentes para não deixar "buracos" no array.

Limitações da Estrutura Estática

- Limite físico do array: 100 peças e 50 serviços, sempre pré-alocados, independentemente de quantos registros existem de fato.
 - Ao tentar cadastrar a 101ª peça, o sistema retorna a mensagem "Array cheio ou lista cheia" em vez de aceitar o novo registro.
- Custo da busca linear: sem índice ordenado, encontrar um registro exige percorrer o array elemento por elemento — complexidade $O(n)$.
 - Update e Delete dependem dessa mesma busca, herdando o mesmo custo.
 - A remoção prefere reorganizar os elementos subsequentes (em vez de exclusão lógica), evitando "buracos", mas pagando o custo de deslocar itens.

```
public static int buscarPeca(int codigo){ 2 usages  Vitor Hugo +1
    //varre o vetor procurando
    for (int i = 0; i < proximoIndex; i++){
        if (pecas[i].codigo == codigo)
            return i;
    }
    return -1;
}
```

Considerações Finais

- Vantagens: acesso a qualquer posição é $O(1)$, já que o índice aponta direto ao endereço na memória contígua.
 - Estrutura simples de implementar e prever em termos de uso de memória, com boa performance de leitura sequencial.
- Desvantagens: o tamanho fixo não se adapta ao crescimento real da base de clientes da MecâniQA.
 - Busca, atualização e remoção custam $O(n)$ por dependerem de varredura linear.
- Esses limites motivam a evolução para estruturas dinâmicas nas próximas sprints do programa de trainee.

Referências

- MecâniQA - Automotive Tech. Programa de Trainee 2026.2: Como trabalhamos, papéis e avaliação.
- Relatórios de Mesa — OAT 1: Explorando Estruturas de Dados Estáticas (28/07, 04/08 e 11/08/2026).
- Caderno de Guias — OAT 1: Tutoriais.
- Barema de Avaliação — Estrutura de Dados, 2026.2.